Good — picking the quantile is a *targeted* choice: it should match the question you want to answer. Below I'll explain the logic, practical trade-offs, recommended defaults, and give a small Python recipe you can drop into your workflow.

1) Concept / mapping: quantile ↔ probability

Quantile regression at quantile `q` estimates the time `t_q` such that

$P(T \le t_q \mid X) = q$.

So:

- If you want the **time that 50% of runners will beat** → use `q = 0.5` (median).

- If you want the **time such that 90% finish by it** → use `q = 0.9`.

- If your goal is "what UTMB index gives me a 75% chance to finish under 27h?" you want to work with the `q = 0.75` predictive curve and invert it (or find UTMB where predicted `t_0.75 <= 27`).

So pick `q` equal to the **probability of success** you care about.

2) Practical tradeoffs / cautions

- **Extreme quantiles (0.95, 0.99)** are *harder to estimate* reliably because very few observations live in the tail. They tend to have higher variance and wider uncertainty.

- **Sample size matters**: if you have thousands of finishers, you can estimate 0.9 reasonably; with a few hundred it gets noisy.

- **Stability:** always check uncertainty via bootstrap or analytic standard errors.

- **Multiple quantiles**: it's often useful to fit several quantiles (e.g. 0.1, 0.25, 0.5, 0.75, 0.9) to see how effects vary across the distribution.

3) Recommendation (what to pick)

- For a single answer about a likely target: pick `q` equal to the desired probability.

- Example: "I want a 0.8 (80%) chance to finish before 27 h" → fit q = 0.8 and solve for UTMB index that gives `t_0.8 = 27`.

- For exploratory / reporting: fit a small grid of quantiles such as `[0.1, 0.25, 0.5, 0.75, 0.9]`. This shows the whole conditional distribution and how covariate effects change.

- Avoid trying to use tiny-sample extreme quantiles unless you also bootstrap heavily.

4) How to communicate results in the blog / to a runner

- “The 90% quantile regression gives the finishing time such that 90% of similar runners finish faster. If you want a 90% chance to be under 27h, find `UTMB_index` such that the model’s 0.9-quantile prediction equals 27h.”

- Report uncertainty: e.g. “Predicted 90% time = 27h (95% CI: 26.0–28.0)”.

5) Quick Python recipe (statsmodels) — fit multiple quantiles + CI via bootstrap

```python
import numpy as np
import pandas as pd
from statsmodels.regression.quantile_regression import QuantReg

# data: aft_data_finishers (ONLY finishers, since this is conditional on finishing)
# y = runner_race_time_hours, X = design matrix (with intercept)

y = aft_data_finishers['runner_race_time_hours']
X = sm.add_constant(aft_data_finishers[['runner_overall_index','runner_age','gender_Male']])

quantiles = [0.1, 0.25, 0.5, 0.75, 0.9]
models = {}
for q in quantiles:
mod = QuantReg(y, X).fit(q=q)
models[q] = mod

# predict on new X0 (DataFrame with same columns as X)
X0 = sm.add_constant(pd.DataFrame({
'runner_overall_index': [656],
'runner_age': [28],
'gender_Male': [1]
}))

preds = {q: models[q].predict(X0)[0] for q in quantiles}
```

Bootstrap CIs for a quantile prediction

```python
nboot = 1000
q = 0.9
boot_preds = []
```

```
rng = np.random.default_rng(123)
for i in range(nboot):
idx = rng.choice(len(y), size=len(y), replace=True)
yb = y.iloc[idx]
Xb = X.iloc[idx]
modb = QuantReg(yb, Xb).fit(q=q, max_iter=200)
boot_preds.append(modb.predict(X0)[0])

ci_lower, ci_upper = np.percentile(boot_preds, [2.5, 97.5])
```

6) How to invert quantile regression to get required UTMB index

- Either: solve numerically for `UTMB_index` such that `predicted_time_q(UTMB_index) == target_time` (you can evaluate model prediction across UTMB grid and interpolate), or

- If model is linear in UTMB index for that quantile (it is), you can algebraically invert:

UTMB = (t_target - (β0 + βage*age + βg*gender)) / βindex

where β are quantile regression coefficients for chosen `q`.

Be careful with sign / beta_index near zero.

7) Diagnostics & model quality

- There's no single R^2 equivalent; use:

- **Pseudo-R^2** measures for quantile models (available in output).

- **Check residuals around the quantile** (e.g., proportion of observations below predicted quantile should be ≈ q).

- **Cross-validation**: forecast quantiles on held-out folds and compute pinball loss.

- **Bootstrap for coefficient uncertainty**.

TL;DR

- Choose the quantile `q` to match the probability you care about (e.g., q=0.9 for a 90% guarantee).

- Fit multiple quantiles (0.1,0.25,0.5,0.75,0.9) for a full picture.

- Use bootstrap to get reliable CIs for tail quantiles.

- Invert the quantile model to find required UTMB index for a given target time.